

Assignment 3: Serverless Customer Review Processing

Event-Driven Profanity Check and Sentiment Analysis on MiniStack

Group 48, Data-Intensive Computing, TU Wien, SS 2026

Aaradhaya Arora (12534787) Ekaterina Dobrynina (12332276) Gopinath Hariharasudhan (52202173)
Serhii Slutu (12537831) Sofia Zubkova (12534780)

June 29, 2026

AI disclosure. We used various AI tools (e.g., ChatGPT, Claude via Cursor) as a support tool while working on this assignment. Its use was limited to text polishing (grammar, wording), to debugging help when we were stuck, and to drafting assistance for this report. The serverless architecture, the choice of AWS services and triggers, the Lambda implementations, and all design decisions were made by us; the models did not generate the solution on their own, they were used purely as assistance.

1 Introduction

This assignment asks us to build an event-driven serverless application that ingests customer product reviews and performs three main tasks on each review: text preprocessing, profanity checking, and sentiment analysis. In addition, the system must track how many impolite reviews each customer submits and ban users who exceed a configurable threshold.

We implement the application on *MiniStack*, a local AWS emulator running on the LBD cluster. The design follows the course tutorial: Lambda functions are deployed independently, communicate only through AWS platform events (S3 object creation and DynamoDB updates), and read all resource names from the Systems Manager (SSM) Parameter Store rather than hard-coding them.

The development set `reviews_devset.json` contains 78 829 Amazon-style review records. Each review is uploaded individually to S3, which starts the processing chain. In the rest of this report we describe the problem, our architecture and implementation, and the numbers produced on the development set.

2 Problem Overview

Input. Each review is a JSON object. The fields relevant to this assignment are `reviewText` (the main review body), `summary` (a short headline), `overall` (a numeric star rating), and `reviewerID` (the customer identifier).

Preprocessing. The first Lambda tokenizes the combined text of `summary` and `reviewText`, removes English stop words, POS-tags the remaining tokens, and lemmatizes them with WordNet. The enriched record is written to a processed-reviews S3 bucket.

Profanity check. The second Lambda scans the same text fields for bad words using the `profanityfilter` library, marks each review as `clean` or `impolite`, stores the result in a DynamoDB `reviews` table, and increments an impolite-review counter for the submitting user. When a user's count exceeds the ban threshold (3 impolite reviews, i.e. more than three), the user is marked as banned in a separate `users` table.

Sentiment analysis. The third Lambda runs NLTK's VADER sentiment analyser on the text fields, classifies each review as `positive`, `neutral`, or `negative`, and compares the text sentiment against the numeric `overall` rating to record whether they agree.

Constraints. The assignment requires at least three Lambda functions, an event-driven chain starting from a single S3 upload, SSM for configuration, and automated integration tests. The reported results must be computed from `reviews_devset.json` only.

3 Methodology and Approach

3.1 Architecture

Figure 1 shows the complete function chain and how each component interacts with the AWS platform services. No Lambda invokes another directly; every hand-off is mediated by an S3 or DynamoDB event.

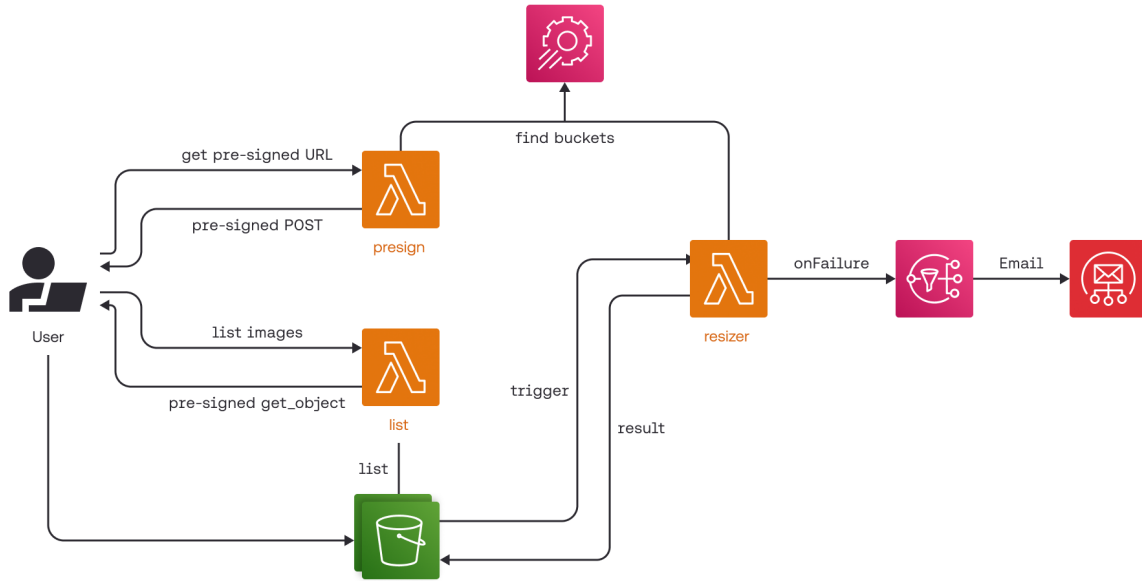


Figure 1: Serverless review-processing architecture. Uploading one review JSON to the raw bucket triggers preprocessing; the resulting object in the processed bucket triggers profanity-check and sentiment-analysis in parallel. Both downstream Lambdas write to DynamoDB. Bucket and table names are resolved at runtime from SSM Parameter Store.

3.2 Trigger chain

1. **Upload.** A driver script (or an integration test) puts one review JSON into the *raw reviews* S3 bucket.
2. **Preprocessing** (`preprocessing` Lambda) is triggered by `s3:ObjectCreated`. It reads the object, preprocesses the text, assigns a `reviewId` (using an optional `sourceId` tag for devset rows), and writes the enriched JSON to the *processed reviews* bucket.
3. **Profanity check** (`profanity-check` Lambda) and **sentiment analysis** (`sentiment-analysis` Lambda) both listen on the processed bucket's `ObjectCreated` event and run *in parallel* on the same object.
4. **DynamoDB writes.** Profanity-check creates or updates a row in the reviews table (setting `profanityCheck` and user metadata) and updates the user's impolite count in the users table. Sentiment-analysis updates the same reviews row with `sentiment`, `sentimentScore`, and `ratingSentimentAgreement`.

Because the two downstream Lambdas can finish in either order, both use DynamoDB `update_item` with `if_not_exists` guards so that neither overwrites fields already written by the other.

3.3 Configuration via SSM

All infrastructure names are stored in SSM and read at Lambda cold start:

Listing 1: SSM parameters used by the application

```
/dic-reviews-app/buckets/raw
/dic-reviews-app/buckets/processed
/dic-reviews-app/tables/reviews
/dic-reviews-app/tables/users
/dic-reviews-app/config/ban-threshold
```

The deployment script `run.sh` creates the buckets and tables, writes these parameters, packages each Lambda (NLTK and `profanityfilter` dependencies bundled as Linux wheels), and registers the S3 notification configurations.

3.4 Libraries and design choices

- **NLTK** for tokenization, POS tagging, lemmatization (preprocessing) and VADER sentiment scoring (sentiment analysis). NLTK data packages are downloaded to `/tmp` on first invocation inside the Lambda container.
- **profanityfilter** for the profanity check. The filter operates on the raw `summary` and `reviewText` strings (not on the lemmatized tokens), which keeps behaviour aligned with the assignment’s intent.
- **Parallel S3 triggers** for profanity-check and sentiment-analysis. An earlier design used a DynamoDB Stream on the reviews table to trigger sentiment analysis, but under high load on MiniStack the stream backlog caused timeouts when processing the full devset. Triggering both functions from the processed bucket is still fully compliant with the assignment requirement that invocations be driven by S3 and/or DynamoDB events.
- **Deterministic devset IDs.** The upload script tags each devset review with `sourceId = devset-NNNNNN`, which preprocessing propagates as `reviewId`. This lets the tally script isolate devset-derived rows from ad-hoc test uploads without extra bookkeeping.

3.5 Testing

Integration tests in `tests/test_integration.py` upload synthetic reviews to the raw bucket and poll DynamoDB until the expected state appears. They cover:

- a clean, positive review flowing through all three stages;
- a rating/sentiment disagreement (5-star rating with negative text);
- profanity detection (`damn` in the review text);
- automatic banning after four impolite reviews from the same user (threshold is `> 3`).

To produce the assignment results, `scripts/run_devset.py` uploads every review from `reviews_devset.json` individually, and `scripts/tally_results.py -wait` polls until all devset rows have a final sentiment label.

4 Results

We processed the full development set on MiniStack after deploying with `run.sh`. Table 1 summarises the numbers; they match the output written to `data/results_summary.json`.

Table 1: Results on `reviews_devset.json` (78 829 reviews).

Metric	Count
Total reviews processed	78 829
Positive sentiment	67 522
Neutral sentiment	1 573
Negative sentiment	9 734
Reviews failing profanity check	3 135

Banned users. One user from the development set exceeded the ban threshold: `A320TMDV6KCFU` submitted four impolite reviews (`impoliteCount` = 4) and was marked `banned = true` in the users table.

Rating vs. text sentiment. For every devset review that carried an `overall` rating, we also recorded whether the VADER label agreed with the rating bucket ($\text{rating} \leq 2 \rightarrow \text{negative}$, $\geq 4 \rightarrow \text{positive}$, otherwise neutral). Across 78 829 compared reviews, 15 469 showed a disagreement between the star rating and the text sentiment. This is a useful sanity check that the `overall` field is being taken into account as required.

5 Conclusions

We built a three-Lambda serverless pipeline on MiniStack that preprocesses, profanity-checks, and sentiment-analyses customer reviews in a fully event-driven way. S3 uploads start the chain; parallel S3 triggers fan out profanity checking and sentiment analysis; DynamoDB stores the final state and tracks user bans. All configuration is externalised to SSM, and integration tests verify the main behaviours.

On the 78 829-review development set the pipeline classified the majority of reviews as positive (67 522), flagged 3 135 as impolite, and banned one repeat offender. The implementation re-deploys cleanly after every MiniStack restart via `run.sh`, which matches the ephemeral nature of the emulator environment described in the assignment.